

Annexe A13 bis — Le LOD, de fond en comble

LibraryV3 · Phase G, chantier 1 · complète et amende l'annexe A13 (sélection) · septembre 2026

L'annexe A13 a posé les formules de sélection : erreur géométrique ϵ mesurée hors ligne, facteur de projection q , seuils dérivés d'une tolérance τ en pixels. Ce document prend le sujet par l'autre bout : ce qu'est vraiment un LOD, quand il se justifie, comment on fabrique les niveaux, à quel moment on les calcule, et surtout comment on range les données pour un CPU. Il amende A13 § 6.4 sur un point (§ 6).

1. Un LOD est un filtre avant l'échantillonnage

L'écran est une grille d'échantillonnage : le rasterizer de LV3 ne teste qu'un point par pixel, son centre. Tout détail géométrique plus fin que l'espacement entre deux centres est soit invisible, soit instable : une bosse touche un centre à une frame, le rate à la suivante. C'est de l'aliasing géométrique, le scintillement des objets lointains trop détaillés.

Le remède vient du traitement du signal : **filtrer avant d'échantillonner**. Les mipmaps pré-filtrent les textures ; le LOD pré-filtre la géométrie. Un niveau grossier est une version du maillage dont les détails ont la taille que l'écran peut encore montrer. Le LOD a donc deux bénéfices :

- **Performance** : on ne paie plus des faces qui ne produisent rien. Dans LV3, $\text{Render} \approx 18,7 \text{ ns} \times \text{FacesSubmitted}$, que la face couvre 0 ou 500 pixels.
- **Qualité** : moins de scintillement, car on ne demande plus à la grille de représenter ce qu'elle ne peut pas voir.

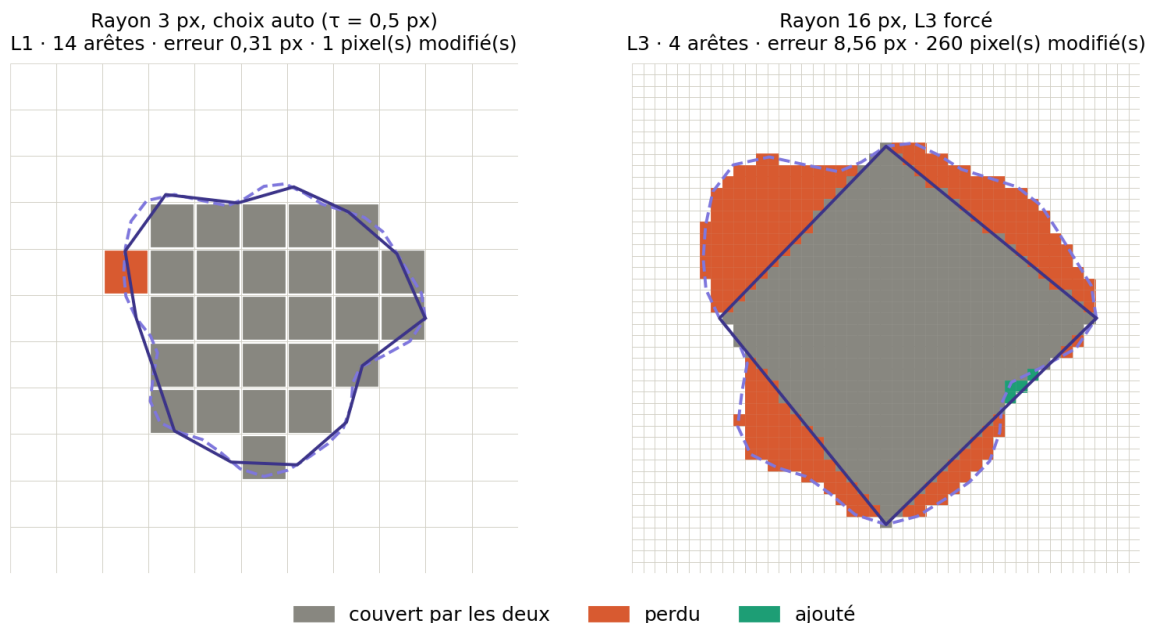


Figure 1 — Analogie 2D : silhouette d'un rocher rasterisée au centre des pixels. À gauche, à 3 px de rayon, la sélection automatique descend à L1 (14 arêtes au lieu de 40) pour un seul pixel modifié. À droite, le niveau L3, parfait à très petite taille, est grossièrement faux à 16 px. Le pointillé est la silhouette de référence L0.

Trois enseignements de la figure :

- À petite taille, la majorité des faces de L0 ne change rien à l'image : c'est le cœur du LOD.
- Le même niveau peut être juste à 2 px et faux à 16 px. **Le niveau n'est pas une propriété de l'objet, mais de l'objet dans une vue.**
- τ est le seul bouton de qualité du système ; tout le reste se déduit des mesures.

2. Quand s'en servir : le bilan coût / gain

2.1 Le gain

Par paire (instance, vue), le gain vaut $(F_0 - F_k) \times 18,7$ ns. Un rocher passant de L0 à L3 évite 160 faces, soit environ 3 μ s par vue. La sélection coûte une MulRow, une division et une à trois comparaisons : quelques nanosecondes (ordre de grandeur, à confirmer par la mesure). La sélection n'est jamais le problème.

2.2 Le vrai critère : les faces par pixel

Le bon indicateur est le rapport **faces soumises / pixels couverts**. Un astéroïde de la ceinture soumet 168 faces pour environ 3,5 pixels : près de 50 faces par pixel. Une planète vue de près soumet quelques milliers de faces sur des dizaines de milliers de pixels : moins d'une face par pixel, le LOD n'a rien à lui retirer. Un objet est candidat quand il réunit :

1. beaucoup de faces par rapport à sa taille habituelle à l'écran ;
2. beaucoup d'instances (le gain se multiplie) ;
3. une forme qui se simplifie bien (organique, arrondie, sans arêtes vives significantes).

2.3 Quand ne pas s'en servir

- **Objets toujours proches** (cockpit, arme en vue FPS) : ils ne quittent jamais L0, la chaîne ne coûte que de la mémoire.
- **Meshes déjà minimaux** : un cube de 12 faces est son propre plancher.
- **Gizmos et outils de debug** : un instrument ne doit pas se dégrader, sinon il ment (A13 § 7.4).
- **Coût dominé par les pixels** : le LOD réduit les bandes sommet et triangle, pas le remplissage. Dans LV3, les pixels ne sont pas le goulot aujourd'hui ($\approx 14\,372$ testés par frame sur la ceinture).

2.4 Le niveau « Culled »

Unity propose un dernier état où l'objet n'est plus dessiné sous une certaine taille. Ce n'est plus du LOD au sens strict : l'image change, des roches de 1 px disparaissent. C'est une décision de contenu, avec son propre critère, désactivée par défaut et jamais mélangée à τ .

3. Construire les niveaux

Un niveau est un maillage fermé complet, jamais un sous-ensemble troué du précédent. Trois familles de méthodes :

3.1 Ré-échantillonner à la source

Quand l'objet vient d'une fonction (rock() de gen_meshes.py, une sphère UV), on la rappelle avec une grille plus grossière. C'est la méthode actuelle de LV3 et la meilleure quand elle est possible : exacte, gratuite, sans artefact. Limite : les sommets d'un niveau ne sont pas forcément ceux du niveau précédent (L1 du rocher échantillonne à 45°, L0 à 30°). La chaîne n'est donc pas *emboîtée*, ce qui compte au § 5.

3.2 Regroupement de sommets (vertex clustering)

Une grille 3D est posée sur l'objet ; tous les sommets d'une cellule fusionnent. Très rapide, robuste, mais laid : la topologie change au hasard. Réservé aux fonds très lointains.

3.3 Effondrement d'arêtes guidé par quadriques (QEM, Garland et Heckbert 1997)

1. Chaque face définit un plan ; la distance au carré d'un point p à ce plan s'écrit $p^T K p$, avec K une matrice 4×4 .
2. Chaque sommet accumule la somme Q des K de ses faces voisines.
3. Chaque arête (u, v) reçoit un coût : fusionner u et v en p coûte $p^T (Q_u + Q_v) p$.
4. Les arêtes vont dans une file de priorité ; on effondre la moins chère, on met à jour les voisines, on recommence jusqu'au nombre de faces visé.
5. On protège ce qui se voit : bords, coutures d'UV et de normales, et on refuse tout effondrement qui retournerait une face.

Deux variantes pour p . Le **point optimal** (système 3×3) donne la meilleure qualité mais crée des sommets. Le **demi-effondrement** ($p = u$ ou $p = v$) ne crée aucun sommet : chaque niveau utilise un sous-ensemble des sommets du précédent, la chaîne est **emboîtée**.

Attention : le coût QEM n'est pas la distance de Hausdorff. C'est un guide pour ordonner les effondrements. L'erreur ϵ stockée se mesure après coup, par la méthode d'A13 § 4. Le simplificateur propose, la mesure décide.

3.4 Combien de niveaux

On suit la courbe faces / erreur (A13 § 4.5) : un facteur 2 à 4 sur les faces entre deux niveaux, et on s'arrête quand le suivant n'apporte plus rien.

4. À quel moment calculer les niveaux

Moment	Qui calcule	Exemple
Création	l'artiste, dans un outil externe	Unity LOD Group : niveaux modélisés à la main
Import / build	l'outil du moteur, une fois ; résultat stocké dans l'asset	Unity Mesh LOD, génération automatique d'Unreal
Chargement	le moteur, à chaque démarrage	rare, meshes procéduraux générés par le moteur
Par frame	interdit	—

Même quand un moteur professionnel « calcule les niveaux à partir du mesh fourni », il le fait à l'import, dans l'éditeur, et stocke le résultat. Au lancement, il charge des niveaux déjà prêts.

Choix LV3 : hors ligne, dans `gen_meshes.py`, qui joue le rôle de l'étape de build. Le moteur reçoit plusieurs fichiers déjà dégradés et un descripteur de chaîne. Simplifier au chargement dans la LIB est écarté pour quatre raisons : le code (un QEM correct n'a rien à faire dans un rasterizer), le temps (la mesure de ϵ est en $O(M \times F)$, payée à chaque démarrage pour un résultat identique), le contrôle (un niveau hors ligne s'inspecte et se versionne dans git), la cohérence (`rock()` ne vit que dans le script).

5. Stocker : le cœur DOD

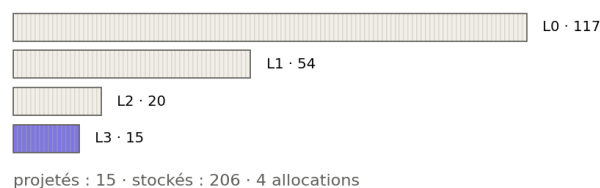
Règle d'A12 : on range chaque donnée selon sa fréquence de changement et selon qui la lit. Un système de LOD a cinq couches.

Couche	Contenu	Change	Lu par	Où
Asset	géométrie des niveaux, ϵ_k , AABB union	jamais	le dessin	ResourceManager, froid
Table de sélection	$\text{invEps}_k = 1 / \epsilon_k$, nombre de niveaux	au chargement	la sélection	tableau dense, chaud
Instance	handle vers la chaîne	rarement	culling, sélection	MeshComponent
Vue	kPx / τ , $ c $, near	chaque frame	la sélection	ViewData
(instance, vue)	q , k	chaque frame	consommé aussitôt	registres, jamais stocké

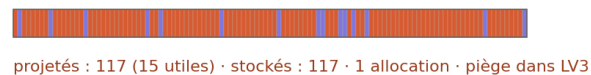
- **Le composant référence la chaîne, jamais le niveau.** Un mesh sans LOD est une chaîne de longueur 1 : aucun cas particulier, aucun nouveau composant.
- **Séparer le chaud du froid.** La sélection ne lit que invEps (16 octets pour 4 niveaux) ; le dessin ne lit que la plage du niveau retenu.
- **τ appartient à la vue**, pas à l'asset (le « LOD Bias » d'Unity est ce multiplicateur global).

5.1 Où ranger les sommets des niveaux

A un mesh par niveau



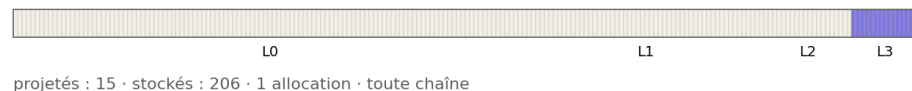
B tampon partagé



C partagé, ordonné en préfixe



D concaténé, plages disjointes



Chaîne du rocher, niveau L3 sélectionné

Figure 2 — Quatre dispositions des sommets de la chaîne du rocher (117, 54, 20, 15 sommets), niveau L3 sélectionné.

5.2 Le piège de la disposition B dans LV3

La disposition B est celle d'Unity Mesh LOD (6.2) : un seul tampon de sommets, des plages d'indices par niveau. C'est excellent sur GPU, où le vertex shader ne traite que les sommets référencés. Mais dans LV3, RasterSpaceBuffer et ClipSpaceBuffer sont complets par instance

(A12a) : ProjectPositions projette tout le tampon. À L3, 117 sommets projetés pour 15 utiles ; la bande sommet ne gagne rien.

Une structure de données n'est optimale que par rapport à la machine qui la lit. Copier un moteur professionnel sans copier son matériel, c'est importer ses compromis sans ses bénéfices.

5.3 Recommandation

- **Maintenant : A**, un MeshClass par niveau regroupés par une chaîne. Rien à changer dans le pipeline.
- **Ensuite : D**, une allocation par chaîne avec une plage {début, nombre} de sommets et d'indices par niveau (modèle Unreal). Coût : la somme des niveaux, ici +76 % de sommets, quelques kilo-octets.
- **Plus tard, avec un générateur par demi-effondrement : C**, sommets ordonnés du plus grossier au plus fin ; le niveau k projette un préfixe. Réservé aux chaînes emboîtées ; le rocher actuel ne l'est pas.

6. Le ResourceManager et l'amendement à A13 § 6.4

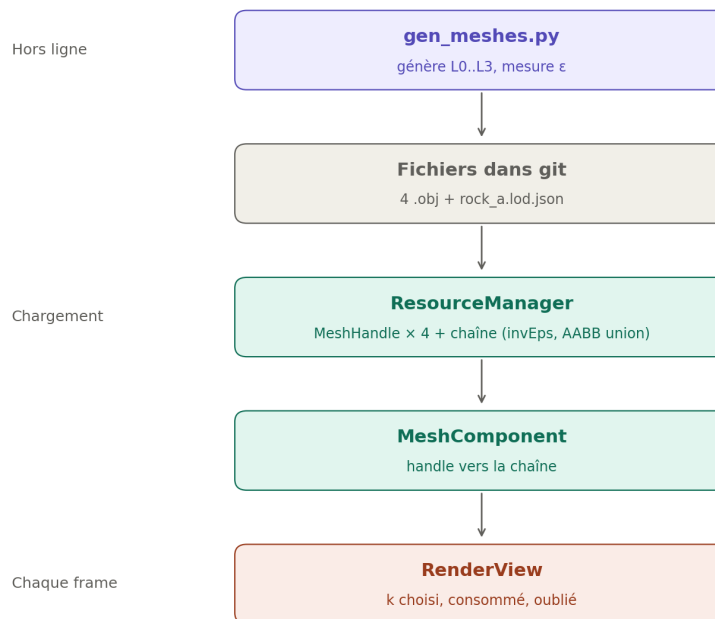


Figure 3 — Trajet des niveaux, du script de génération à la sélection par vue.

Le ResourceManager fait deux choses. Il charge chaque niveau comme un mesh ordinaire via LoadMeshChecked (normalisation de chemin et déduplication de la leçon 3) : un niveau reste un MeshClass avec son MeshHandle, et peut servir à la fois dans une chaîne et seul, sans double chargement. Il construit un objet nouveau, la **chaîne**, à partir d'un descripteur .lod.json : handles des niveaux, invEps, AABB union de tous les niveaux (A13 § 7.2). Le MeshComponent porte le handle de la chaîne ; le Serializer lit un chemin de chaîne, et un .obj seul devient une chaîne de longueur 1. Le composant référence, il ne possède rien.

Amendement à A13 § 6.4. A13 précalculait $q\text{Max}_k = \tau / \epsilon_k$ au chargement, dans la chaîne. C'est incompatible avec τ par vue : une chaîne partagée par 4 vues ne peut contenir un seuil dépendant de la vue. Correction : la chaîne stocke $\text{invEps}_k = 1 / \epsilon_k$; la vue calcule une fois par

frame kPx / τ ; la sélection compare $q' \leq \text{invEps}_k$ avec $q' = q / \tau$. La division par τ est absorbée dans une constante de vue ; la boucle chaude reste faite de comparaisons pures.

Question ouverte : le déchargement. Un niveau partagé entre une chaîne et un usage autonome ne doit pas être détruit avec la chaîne. `UnloadMesh` n'étant appelée nulle part (leçon 3 § 9.1), la question sera tranchée avec un futur système de niveaux de jeu.

7. Exécution et transitions

La sélection vit dans `RenderView`, par (instance, vue), après le frustum culling et avant la bifurcation `Inside / Intersect` (A13 § 7.3). Évolution possible, **après mesure seulement** : deux passes par vue, sélection de toutes les instances visibles, puis dessin regroupé par mesh pour garder les petits maillages chauds en L1.

Si l'hystérésis devient nécessaire (A13 § 9), le niveau précédent se mémorise par (instance, vue) : un tableau d'un octet par instance dense, un par vue, possédé par la vue ($\approx 2,5$ Ko). Jamais dans le composant, partagé par les 4 vues.

8. Unity, Unreal et LV3

	Unity LOD Group	Unity Mesh LOD	Unreal Static Mesh	LV3
Chaîne	objets enfants	dans le mesh	dans l'asset	dans l'asset
Sommets	un tampon par niveau	tampon partagé	un par niveau	A puis D
Critère	hauteur relative écran	taille écran	taille écran	$\varepsilon \cdot q \leq \tau$ en pixels
Réglage global	LOD Bias	réglages projet	biais plateforme	τ par vue

LV3 dérive ses seuils d'une erreur mesurée au lieu d'un seuil réglé à l'œil. C'est plus strict, et c'est ce qui a démasqué l'hypothèse H1 (A13 § 11).

9. Contre-exemples

X Reproduire le LOD Group d'Unity dans l'ECS : une entité enfant par niveau activée et désactivée chaque frame. Écritures dans le monde, drapeaux invalidés, multi-vues impossible. Le niveau est un résultat de calcul, pas un état de scène.

X Tampon partagé avec projection de tout le tampon : la mémoire baisse, la bande sommet ne gagne rien.

X Stocker ε multiplié par l'échelle d'une instance : faux pour toutes les autres. ε vit en unités locales.

X Prendre le coût QEM pour l'erreur du niveau : il ordonne le travail, il ne garantit rien. On mesure ε après.

X Précalculer τ / ε dans la chaîne alors que τ dépend de la vue (§ 6).

X Activer « Culled » sous le nom de LOD : faire disparaître des objets change l'image.

X Choisir le niveau dans `WorldTransformSystem` : mauvaise fréquence, il ne connaît ni la vue ni la projection.

X Simplifier au chargement dans la LIB : code lourd, coût payé à chaque démarrage, résultat invisible et non versionné.

10. Plan du chantier 1

1. Remettre LV3_LOD_STATS à 0 par défaut dans Core/config.h (dette signalée en C4), puis faire le run de comptage des tailles apparentes.
2. Écrire la règle de décision avant la mesure (M12).
3. Étendre gen_meshes.py : chaîne du rocher, ϵ mesurés, descripteur .lod.json.
4. Introduire la chaîne (disposition A) dans le ResourceManager, le handle de chaîne dans MeshComponent, la sélection dans RenderView, puis mesurer.